

찾기

시간 제한	메모리 제한	제출	정답
2 초	256 MB	52390	16766

문제

워드프로세서 등을 사용하는 도중에 찾기 기능을 이용해 본 일이 있을 것이다. 이 기능을 여러분이 실제로 구현해 보도록 하자.

두 개의 문자열 P 와 T 에 대해, 문자열 P 가 문자열 T 중간에 몇 번, 어느 위치에서 나타나는지 알아내는 문제를 '문자열 매칭'이라고 한다. 워드프로세서의 찾기 기능은 이 문자열 매칭 문제를 풀어주는 기능이라고 할 수 있다. 이때의 P 는 패턴이라고 부르고 T 는 텍스트라고 부른다.

편의상 T 의 길이를 n , P 의 길이를 m 이라고 하자. 일반적으로, $n \geq m$ 이라고 가정해도 무리가 없다. $n < m$ 이면 어차피 P 는 T 중간에 나타날 수 없기 때문이다. 또, T 의 i 번째 문자를 $T[i]$ 라고 표현하도록 하자. 그러면 물론, P 의 i 번째 문자는 $P[i]$ 라고 표현된다.

1 2 3 4 5 6 7 8 9 ...

$T : [A B C D A B C D A B D E]$

| | | | | X

$P : [A B C D A B D]$

1 2 3 4 5 6 7

문자열 P 가 문자열 T 중간에 나타난다는 것, 즉 문자열 P 가 문자열 T 와 매칭을 이룬다는 것이 어떤 것인지 위와 아래의 두 예를 통해 알아보자. 위의 예에서 P 는, T 의 1번 문자에서 시작하는 매칭에 실패했다. T 의 7번 문자 $T[7]$ 과, P 의 7번 문자 $P[7]$ 이 서로 다르기 때문이다.

그러나 아래의 예에서 P 는, T 의 5번 문자에서 시작하는 매칭에 성공했다. T 의 5~11번 문자와 P 의 1~7번 문자가 서로 하나씩 대응되기 때문이다.

1 2 3 4 5 6 7 8 9 ...

$T : [A B C D A B C D A B D E]$

|||||||

P : [A B C D A B D]

1 2 3 4 5 6 7

가장 단순한 방법으로, 존재하는 모든 매칭을 확인한다면, 시간복잡도가 어떻게 될까? T의 1번 문자에서 시작하는 매칭이 가능한지 알아보기 위해서, T의 1~m번 문자와 P의 1~m번 문자를 비교한다면 최대 m번의 연산이 필요하다. 이 비교들이 끝난 후, T의 2번 문자에서 시작하는 매칭이 가능한지 알아보기 위해서, T의 2~m+1번 문자와 P의 1~m번 문자를 비교한다면 다시 최대 m번의 연산이 수행된다. 매칭은 T의 n-m+1번 문자에서까지 시작할 수 있으므로, 이러한 방식으로 진행한다면 $O((n-m+1) \times m) = O(nm)$ 의 시간복잡도를 갖는 알고리즘이 된다.

더 좋은 방법은 없을까? 물론 있다. 위에 제시된 예에서, $T[7] \neq P[7]$ 이므로 T의 1번 문자에서 시작하는 매칭이 실패임을 알게 된 순간으로 돌아가자. 이때 우리는 매칭이 실패라는 사실에서, $T[7] \neq P[7]$ 라는 정보만을 얻은 것이 아니다. 오히려 $i=1...6$ 에 대해 $T[i] = P[i]$ 라고 하는 귀중한 정보를 얻지 않았는가? 이 정보를 십분 활용하면, $O(n)$ 의 시간복잡도 내에 문자열 매칭 문제를 풀어내는 알고리즘을 설계할 수 있다.

P 내부에 존재하는 문자열의 반복에 주목하자. P에서 1, 2번 문자 A, B는 5, 6번 문자로 반복되어 나타난다. 또, T의 1번 문자에서 시작하는 매칭이 7번 문자에서야 실패했으므로 T의 5, 6번 문자도 A, B이다.

따라서 T의 1번 문자에서 시작하는 매칭이 실패한 이후, 그 다음으로 가능한 매칭은 T의 5번 문자에서 시작하는 매칭임을 알 수 있다! 더불어, T의 5~6번 문자는 P의 1~2번 문자와 비교하지 않아도, 서로 같다는 것을 이미 알고 있다! 그러므로 이제는 T의 7번 문자와 P의 3번 문자부터 비교해 나가면 된다.

이제 이 방법을 일반화 해 보자. T의 i번 문자에서 시작하는 매칭을 검사하던 중 $T[i+j-1] \neq P[j]$ 임을 발견했다고 하자. 이렇게 P의 j번 문자에서 매칭이 실패한 경우, $P[1...k] = P[j-k...j-1]$ 을 만족하는 최대의 $k(\neq j-1)$ 에 대해 T의 i+j-1번 문자와 P의 k+1번 문자부터 비교를 계속해 나가면 된다.

이 최대의 k를 j에 대한 함수라고 생각하고, 1~m까지의 각 j값에 대해 최대의 k를 미리 계산해 놓으면 편리할 것이다. 이를 전처리 과정이라고 부르며, $O(m)$ 에 완료할 수 있다.

이러한 원리를 이용하여, T와 P가 주어졌을 때, 문자열 매칭 문제를 해결하는 프로그램을 작성하시오.

입력

첫째 줄에 문자열 T가, 둘째 줄에 문자열 P가 주어진다. T와 P의 길이 n, m은 1이상 100만 이하이고, 알파벳 대소문자와 공백으로만 이루어져 있다.

출력

첫째 줄에, T 중간에 P가 몇 번 나타나는지를 나타내는 음이 아닌 정수를 출력한다. 둘째 줄에는 P가 나타나는 위치를 차례대로 공백으로 구분해 출력한다. 예컨대, T의 $i \sim i+m-1$ 번 문자와 P의 1 ~ m번 문자가 차례로 일치한다면, i를 출력하는 식이다.

예제 입력 1

ABC ABCDAB ABCDABCDABDE

ABCDABD

예제 출력 1

1

16

출처

- 문제를 번역한 사람: [author10](#)
- 문제의 오타를 찾은 사람: [zakelstorm](#)
- 잘못된 데이터를 찾은 사람: [lja9702](#)
- 데이터를 추가한 사람: [choyi0521](#), [doju](#), [ik_e](#), [sait2000](#), [Utenie](#), [willook](#), [zkoong](#)

알고리즘 분류

- [문자열](#)
- [kmp](#)